

Phát hiện đối tượng trong ảnh

YOLOv8, YOLOv11, và các mô hình hiện đại.

Thanh-Sach LE

✉ Itsach@hcmut.edu.vn

Faculty of Computer Science and Engineering
Ho Chi Minh City University of Technology
Ho Chi Minh City. Ngày 10 tháng 4 năm 2026

Mục lục — phần 1

1 Giới thiệu

- Định nghĩa và Khái niệm
- Taxonomy

2 Họ Yolo-Detector

- Tổ chức dữ liệu và nhãn
- Yolo: Ý tưởng thiết kế mô hình
- Đồ thị tính toán và kích thước tensor
- Chi tiết đầu phát hiện
- TaskAlignedAssigner
- Loss
- Ultralytics: YOLOv8 và YOLO11

3 Bước suy diễn (Inference)

- Quy trình suy diễn
- Non-Maximum Suppression



Phát hiện đối tượng
trong ảnh

1 Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Mục lục — phần 2

4 Đánh giá kết quả

- Khớp dự đoán và TP/FP/FN
- Precision và Recall
- AP và mAP
- Các lỗi thường gặp
- Dữ liệu và nhãn
- Pipeline dữ liệu

5 Mô hình nâng cao

- NMS-free: YOLOv26
- DETR - Detection Transformer
- Hungarian Matching

6 Tổng kết



Phát hiện đối tượng
trong ảnh

2 Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Giới thiệu

Phát hiện đối tượng
trong ảnh

Mục lục

3 Giới thiệu

Khái niệm

Taxonomy

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Object detection là gì?



Định nghĩa

Object detection dự đoán **vật thể** gì tồn tại và **nó ở đâu** trong ảnh.

$$\hat{\mathcal{Y}} = \{(\hat{b}_i, \hat{c}_i, \hat{s}_i)\}_{i=1}^N$$

- $\hat{b}_i$: bounding box.
- $\hat{c}_i$: lớp (class).
- $\hat{s}_i$: độ tin cậy (confidence).
- N là biến số: không biết trước khi inference.

Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

4 Khái niệm

Taxonomy

Họ Yolo-Detector

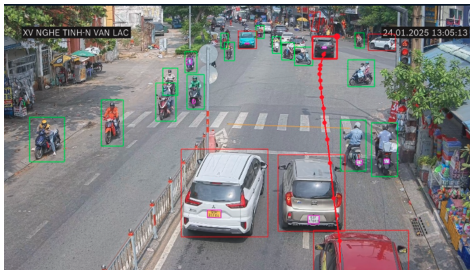
Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Ví dụ minh họa



Chú thích

- Kết quả phát hiện: vị trí (box); loại (màu sắc box); độ tin cậy - không được hiển thị.
- Kết quả tác vụ khác:
 - Phát hiện vi phạm: dấu tròn góc trên phải cho xe vi phạm.
 - Theo vết xe: vết đỏ = quỹ đạo di chuyển của xe theo thời gian.

Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

5 Khái niệm

Taxonomy

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Classification vs Detection vs Segmentation



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

6 Khái niệm

Taxonomy

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Task	Output type	Chi tiết không gian	Metric phổ biến
Classification	Một nhãn/ảnh	Không có	Accuracy
Detection	Boxes + nhãn	Mức đối tượng	mAP
Segmentation	Nhãn pixel	Mức pixel	mIoU / Dice

Ứng dụng thực tế

- **CCTV**: cảnh báo người/phương tiện.
- **Bán lẻ**: phân tích kệ hàng, tự động hóa thanh toán.
- **Giao thông**: phát hiện vi phạm, ước tính lưu lượng.
- **Robotics**: gắp vật thể và tương tác với vật cản.
- **Y tế**: định vị tổn thương/dụng cụ.



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

7 Khái niệm

Taxonomy

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Bounding box formats: xyxy vs cxcywh

$$\text{xyxy} = (x_1, y_1, x_2, y_2), \quad \text{cxcywh} = (c_x, c_y, w, h)$$

- xyxy: thuận tiện cho IoU và clipping.
- cxcywh: thuận tiện cho gán nhãn chuẩn hóa và các phép biến đổi.



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

8 Khái niệm

Taxonomy

Họ Yolo-Detector

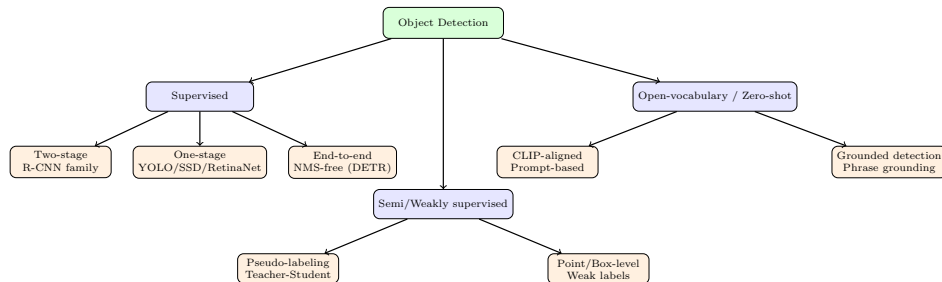
Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Taxonomy các hướng Object Detection



Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Khái niệm

9 Taxonomy

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Chú thích (mô hình tiêu biểu)

- **Two-stage:** Faster R-CNN ([arXiv](#)), Detectron2 ([GitHub](#)).
- **One-stage:** YOLOv8 ([GitHub](#)), RetinaNet ([arXiv](#)).
- **End-to-end NMS-free:** DETR ([arXiv](#)), DINO ([arXiv](#)).
- **Semi/Weakly:** Unbiased Teacher ([arXiv](#)), Soft Teacher ([arXiv](#)).
- **Open-vocabulary:** GLIP ([arXiv](#)), Grounding DINO ([GitHub](#)).

Họ Yolo-Detector

Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

10 Họ
Yolo-Detector

Dữ liệu và nhãn

Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Ví dụ tổ chức file ảnh và nhãn



```
roadsigns12/  
├── data.yaml  
├── test/  
│   ├── images/  
│   │   ├── 020.jpg  
│   │   ├── 021.jpg  
│   │   ├── 022.jpg  
│   │   └── 023.jpg  
│   └── labels/  
│       ├── 020.txt  
│       ├── 021.txt  
│       ├── 022.txt  
│       └── 023.txt  
├── train/  
│   ├── images/  
│   │   ├── 001.jpg  
│   │   ├── 002.jpg  
│   │   ├── 003.jpg  
│   │   └── 004.jpg  
│   └── labels/  
│       ├── 001.txt  
│       ├── 002.txt  
│       ├── 003.txt  
│       └── 004.txt  
├── train.ipynb  
└── valid/  
    ├── images/  
    │   ├── 010.jpg  
    │   ├── 011.jpg  
    │   ├── 012.jpg  
    │   └── 013.jpg  
    └── labels/  
        ├── 010.txt  
        ├── 011.txt  
        ├── 012.txt  
        └── 013.txt
```

Chú thích tổ chức train/val/test

- Mỗi split (train, val, test) tách riêng để tránh rò rỉ dữ liệu.
- Trong từng split: images/ chứa ảnh, labels/ chứa file .txt tương ứng.
- Quy ước tên: mỗi ảnh có đúng 1 file nhãn cùng tên (vd. 001.jpg ↔ 001.txt).
- data.yaml khai báo đường dẫn split và danh sách lớp (names) cho huấn luyện/suy luận.
- Tỷ lệ tham khảo: train lớn nhất; val để chỉnh mô hình; test chỉ dùng đánh giá cuối.

Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

11 Dữ liệu và nhãn

Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

YOLOv8 vs YOLO11

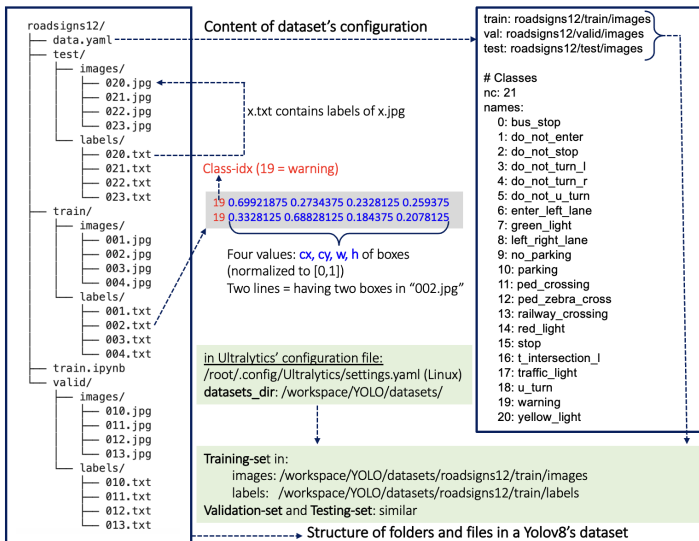
Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Định dạng dữ liệu YOLO



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

12 Dữ liệu và nhãn

Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Nhãn huấn luyện dạng tensor (batch)

- $N_{\max}$: số đối tượng tối đa trong một ảnh sau khi batch hoá — lấy max số box trên từng ảnh trong batch rồi pad các ảnh ít box hơn tới cùng độ dài $N_{\max}$.
- $\text{loc_gt} \in \mathbb{R}^{B \times N_{\max} \times 4}$: tọa độ GT của từng ô trong tensor.
 - **Định dạng điển hình (file nhãn YOLO / Ultralytics)**: mỗi hàng là (c_x, c_y, w, h) chuẩn hoá theo chiều rộng và chiều cao ảnh (mỗi số $\in [0, 1]$): tâm box và kích thước tương đối.
 - Một số pipeline dùng (x_1, y_1, x_2, y_2) chuẩn hoá hoặc tọa độ tuyệt đối pixel — quan trọng là **thống nhất** với bước gán và hàm loss; slide sau trong deck dùng neo ô / offset trên feature map là bước sau khi đã có box GT trong không gian ảnh.
- $\text{cls_gt} \in \mathbb{R}^{B \times N_{\max} \times 1}$ (hoặc $\mathbb{Z}^{B \times N_{\max}}$): chỉ số lớp $0, \dots, N_c - 1$ cho từng slot box (slot pad có thể gán giá trị giả, bị mask loại).
- $\text{mask} \in \mathbb{R}^{B \times N_{\max} \times 1}$ (thường nhị phân hoặc $\{0, 1\}$): 1 = slot có GT hợp lệ, 0 = slot *padding* sau khi căn $N_{\max}$ — dùng để không tính loss / không gán nhãn trên ô giả.

Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

13 Dữ liệu và nhãn

Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

YOLOv8 vs YOLO11

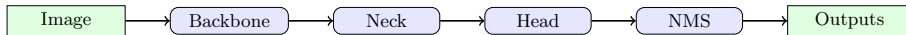
Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Kiến trúc tổng quan



Chú thích

- **Backbone:** trích xuất đặc trưng thị giác đa mức từ ảnh đầu vào (sử dụng CNN).
- **Neck:** hợp nhất đặc trưng đa tỷ lệ (FPN/PAN) để cải thiện phát hiện small/medium/large objects.
- **Head:** dự đoán class logits và box regression cho từng vị trí trên feature maps.
- **NMS:** lựa chọn **hộp giới hạn** bằng **loại bỏ không cực đại** (Non-Maximum Suppression).



Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

14 Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Kiến trúc tổng quan - Backbone



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

15 Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

- Trích xuất đặc trưng thị giác đa mức từ ảnh đầu vào (sử dụng CNN).
- Lấy đầu ra tương ứng các stride khác nhau:
 - P3: stride = 8 $\rightarrow$ (mịn, mạnh cho vật thể nhỏ)
 - P4: stride = 16 $\rightarrow$ (trung gian)
 - P5: stride = 32 $\rightarrow$ (thô, mạnh cho vật thể lớn)
- Ảnh đầu vào: $(B, 3, 640, 640)$.
 - Đầu ra:
 - P3: $(B, C_3, 80, 80)$
 - P4: $(B, C_4, 40, 40)$
 - P5: $(B, C_5, 20, 20)$

Kiến trúc tổng quan - Neck



- Đặc trưng cấp thấp, giữ được nét chi tiết của đối tượng; nhưng lại thiếu ngữ cảnh lớn.
- Đặc trưng cấp cao, giữ được ngữ cảnh lớn; nhưng lại thiếu nét chi tiết của đối tượng.
- Trách nhiệm của Neck: hợp nhất đặc trưng đa tỷ lệ (FPN/PAN) để cải thiện phát hiện small/medium/large objects.
 - **FPN (Feature Pyramid Network):** truyền thông tin ngữ nghĩa từ tầng sâu xuống tầng nông qua cơ chế upsample + merge.
 - **PAN (Path Aggregation Network):** truyền ngược thông tin vị trí/chi tiết từ tầng nông lên tầng sâu qua downsample + merge.
 - Kết quả: mỗi mức đặc trưng vừa có chi tiết cục bộ, vừa có ngữ cảnh mạnh.

Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

16 Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

YOLOv8 vs YOLO11

Bước suy diễn

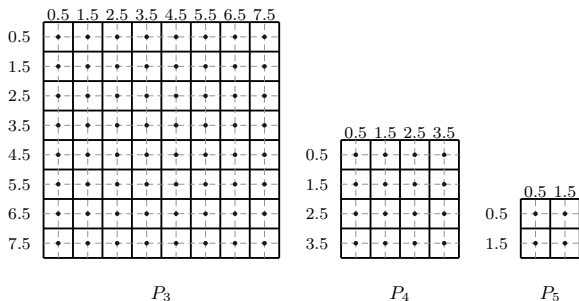
Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Kiến trúc tổng quan - Head

Tạo các điểm tham chiếu - Anchor



Chú thích

- Tại mỗi điểm trên feature map, tạo ra các điểm tham chiếu - **Anchor**.
- $\Rightarrow$ Có bao nhiêu điểm tham chiếu là có bao nhiêu box được dự đoán.
- Vị trí các cạnh của box(left, top, right, bottom) được dự đoán tương đối so với điểm tham chiếu.

Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

17 Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Kiến trúc tổng quan - Head

Cách dự đoán vị trí box



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

18 Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

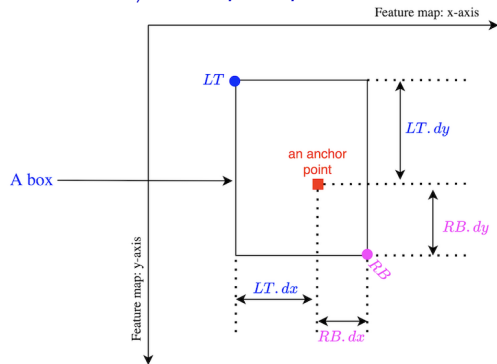
Tổng kết

Dự đoán vị trí box

- Với mỗi box, với mỗi cạnh (left, top, right, bottom), xây dựng mô hình dự đoán phân phối xác suất trên N_r mức lệch so với điểm tham chiếu.
- Mức lệch là các số nguyên từ 0 đến $N_r - 1$
- Ví dụ: $N_r = 16$ thì mức lệch là 0, 1, 2, ..., 15.
- Độ lệch thực sự là kỳ vọng của phân phối xác suất: $\sum_{k=0}^{N_r-1} k \cdot p_k$.

Kiến trúc tổng quan - Head

Bốn độ lệch LT/RB và phân phối trên N_r bin



$$LT.dx = \sum_{x=0}^{N_r-1} (x \times p_x)$$

Displacement of the left-top and the right-bottom corners of the box wrt to the anchor point:

$LT.dx, LT.dy$

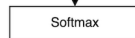
$RB.dx, RB.dy$

One box has 4 the displacements

One displacement needs to be predicted N_r values

Assumption: prediction for $LT.dx$ are as follows:

$[s_0, s_1, s_2, \dots, s_{13}, s_{14}, s_{15}]$



$[p_0, p_1, p_2, \dots, p_{13}, p_{14}, p_{15}]$

p_0 : probability of $LT.dx = 0$

p_1 : probability of $LT.dx = 1$

p_2 : probability of $LT.dx = 2$

$\vdots$

p_{14} : probability of $LT.dx = 14$

p_{15} : probability of $LT.dx = 15$

Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

19 Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Kiến trúc tổng quan - Head

Class logits — YOLOv8 / v11



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

20 Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Dự đoán nhãn phân loại

- Với mỗi box, dự đoán N_c (số lớp) logit cho mỗi lớp.
- Huấn luyện: **sigmoid** + **BCE** (Ultralytics YOLOv8/v11).
- Suy diễn: nhãn bằng **argmax**; ngưỡng **conf** chỉ lọc ô/box yếu.

Kiến trúc tổng quan - NMS



Non-Maximum Suppression (NMS)

- **Vai trò:** chỉ dùng ở bước *suy diễn* (sau huấn luyện; không học được trực tiếp trong loss).
- **Mục đích:** thu gọn danh sách box chồng lấn — giữ các dự đoán mạnh, bỏ bớt trùng lặp cho cùng một vật thể.
- **Cách làm điển hình:** theo lớp, sắp xếp box theo độ tin cậy; lặp lấy box đầu danh sách và **loại bỏ** các box còn lại có chồng IoU $\geq$ ngưỡng (greedy không cực đại — non-max suppression).
- Thường áp sau lọc conf; có biến thể (multiclass-*N*-soft-NMS...) nhưng ý cốt lõi không đổi.

Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

21 Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

YOLOv8 vs YOLO11

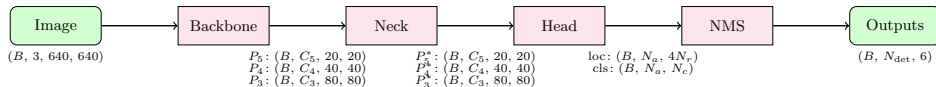
Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Đồ thị tính toán và kích thước tensor (1/2)



Chú thích

- N_a : tổng ô dự đoán trên kim tự tháp (trước NMS); Nếu ảnh vào 640×640 , thì $N_a = 80^2 + 40^2 + 20^2 = 8400$.
- N_r : số mức rời rạc trong DFL ($\text{reg_max} = 16$ thường gặp) $\Rightarrow$ loc có $4 \times N_r$ kênh.
- N_{det} : số box sau NMS / ảnh; phụ thuộc ảnh và ngưỡng.
- N_c : số lớp; cls có N_c kênh.

Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

Yolo: Ý tưởng

22 Kích thước tensor

Chỉ tiết đầu phát hiện

TaskAlignedAssigner

Loss

YOLOv8 vs YOLO11

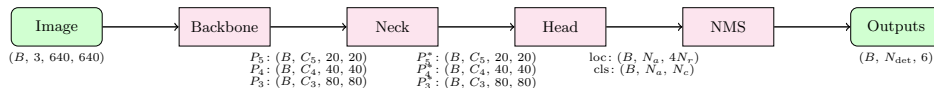
Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Đồ thị tính toán và kích thước tensor (1/2)



Chú thích

- Stride**: $P_3: 8, P_4: 16, P_5: 32$ (diễn hình) — map từ mịn $\rightarrow$ thô.
- Kích thước map**: $h = \lfloor H/s \rfloor, w = \lfloor W/s \rfloor$ (đơn giản $H/s, W/s$ khi chia hết). Ví dụ 640×640 : $80 \times 80, 40 \times 40, 20 \times 20$.

Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

Yolo: Ý tưởng

23 Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

YOLOv8 vs YOLO11

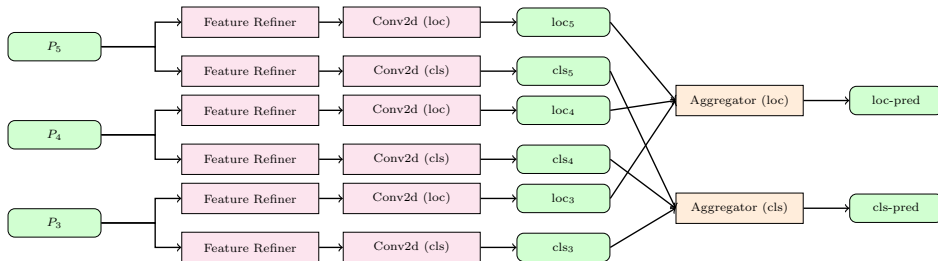
Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Chi tiết đầu phát hiện (1/4)



Chú thích

- **Feature Refiner:** (Conv2d→BatchNorm→SiLU) → (Conv2d→BatchNorm→SiLU).
- **Conv2d(loc):** out-channel = $4 * N_r$, $k=1$, $s=1$, $p=0$.
- **Conv2d(cls):** out-channel = N_c , $k=1$, $s=1$, $p=0$.

Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

Yolo: Ý tưởng

Kích thước tensor

24 Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

YOLOv8 vs YOLO11

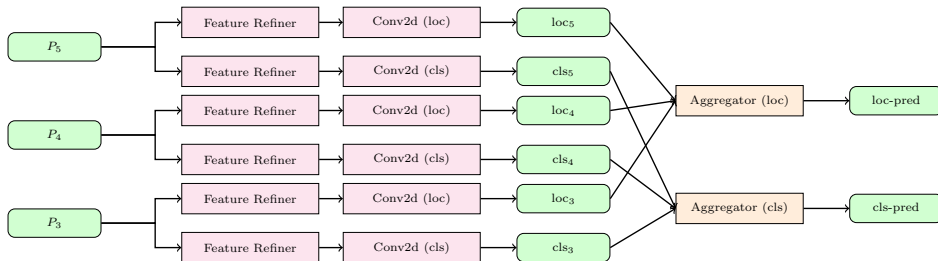
Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Chi tiết đầu phát hiện (2/4)



Chú thích

- loc_5, loc_4, loc_3 : $(B, 4 \times N_r, 20, 20), (B, 4 \times N_r, 40, 40), (B, 4 \times N_r, 80, 80)$.
- loc_pred : $(B, N_a, 4 \times N_r)$.
- cls_5, cls_4, cls_3 : $(B, N_c, 20, 20), (B, N_c, 40, 40), (B, N_c, 80, 80)$.
- cls_pred : (B, N_a, N_c) .

Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

Yolo: Ý tưởng

Kích thước tensor

25 Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Chi tiết đầu phát hiện (3/4)



chỉ số N_a sang features map và vị trí trên map

- Gộp theo thứ tự $P_3 \rightarrow P_4 \rightarrow P_5$, với

$$n_3 = H_3 W_3, \quad n_4 = H_4 W_4, \quad n_5 = H_5 W_5, \quad N_a = n_3 + n_4 + n_5.$$

- Từ chỉ số toàn cục $k \in [0, N_a - 1]$: xác định mức m và chỉ số cục bộ

$$t = \begin{cases} k, & k < n_3 \quad (m = P_3) \\ k - n_3, & n_3 \leq k < n_3 + n_4 \quad (m = P_4) \\ k - n_3 - n_4, & \text{còn lại} \quad (m = P_5) \end{cases}$$

- Đổi t sang ô lưới của mức m : $i = \lfloor t / W_m \rfloor$, $j = t \bmod W_m$.

Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

Yolo: Ý tưởng

Kích thước tensor

26 Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Chi tiết đầu phát hiện (4/4)



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

Yolo: Ý tưởng

Kích thước tensor

27 Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

vị trí trên map -> vị trí trên ảnh

- Với mức m có stride s_m , tâm ô (i, j) trên ảnh đầu vào:

$$x_c = (j + 0.5) s_m, \quad y_c = (i + 0.5) s_m.$$

- Nếu có letterbox/resize, đổi ngược về ảnh gốc:

$$x_{\text{orig}} = \frac{x_c - d_x}{r}, \quad y_{\text{orig}} = \frac{y_c - d_y}{r}.$$

- Kết quả: từ một chỉ số k trong N_a suy ra được mức $P_3/P_4/P_5$, ô map (i, j) , và tâm box trên ảnh thực.

Gán nhãn one-to-many trong YOLO (1/2)

- Mô hình sinh N_a dự đoán rảnh trên P_3, P_4, P_5 . Mỗi ảnh có K hộp GT $\{b_k^{\text{gt}}\}_{k=1}^K$ (lớp c_k).
- One-to-one** (DETR): tìm cặp ghép (i, k) sao cho mỗi k tối đa một i và ngược lại — thường dùng **Hungarian**.
- One-to-many** (YOLO dense): một GT k có thể có **nhiều** chỉ số dự đoán i được coi là **mẫu dương** cùng lúc; các chỉ số khác là nền / âm. Điều này tăng số gradient tích cực trên lưới thưa/dày hơn so với chỉ một ô duy nhất.

Mặt nạ gán (ký hiệu)

Đặt $m_{ik} \in \{0, 1\}$: $m_{ik} = 1$ nếu dự đoán i được gán cho GT k . One-to-many nghĩa là với cùng một k ,

$$\sum_{i=1}^{N_a} m_{ik} \geq 1 \quad \text{và thường có thể} \quad > 1.$$

Ngược lại, Hungarian buộc $\sum_i m_{ik} \leq 1$ và $\sum_k m_{ik} \leq 1$ (khớp một-một toàn cục).

Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

28 TaskAlignedAssigner

Loss

YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Gán nhãn one-to-many trong YOLO (2/2)

- Trong Ultralytics (YOLOv8/YOLO11), **TaskAlignedAssigner** chọn tập dương dựa trên độ “khớp nhiệm vụ” giữa logit lớp và IoU hình học.
- Với dự đoán i và GT k (cùng lớp c_k), điểm căn chỉnh kiểu TOOD/YOLO:

$$t_{ik} = (\sigma(z_{i,c_k}))^\alpha \cdot (\text{IoU}(b_i, b_k^{\text{gt}}))^\beta, \quad \alpha, \beta > 0.$$

- Trên các ứng viên đạt ngưỡng IoU/tiền chọn lọc (theo triển khai), với mỗi GT k giữ **top- r** chỉ số i có t_{ik} lớn nhất $\Rightarrow$ các (i, k) đó có $m_{ik} = 1$ — **một GT, nhiều i** .

Liên hệ loss

Chỉ các i tham gia nhánh localization theo GT được gán mới đóng góp $\mathcal{L}_{\text{box}}/\mathcal{L}_{\text{df}}$; nhánh phân lớp thường gán nhãn nền cho ô không chọn.

Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

29 TaskAlignedAssigner

Loss

YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Loss trong YOLO — tổng quan



- Sau bước gán $\{m_{ik}\}$, loss là tổng có trọng số kiểu huấn luyện một giai đoạn:

$$\mathcal{L} = w_{\text{box}}\mathcal{L}_{\text{box}} + w_{\text{cls}}\mathcal{L}_{\text{cls}} + w_{\text{dfl}}\mathcal{L}_{\text{dfl}}.$$

- $\mathcal{L}_{\text{box}}$, $\mathcal{L}_{\text{dfl}}$: chỉ trên các cặp (i, k) có $m_{ik} = 1$ (và chuẩn hoá theo số mẫu / batch).
- $\mathcal{L}_{\text{cls}}$: BCE đa nhãn trên toàn bộ N_a vị trí — ô dương one-hot theo c_k , ô nền vector không.

Khác một số detector query-based

Không cần Hungarian: gán cục bộ theo t_{ik} và top- $r \Rightarrow$ bản chất **one-to-many** so với matching một-một toàn ảnh.

Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

30 Loss

YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Loss trong YOLO — công thức gọn từng nhánh

- Phân lớp** (sigmoid độc lập từng lớp): với logit $z_{i,c}$, xác suất $\hat{p}_{i,c} = \sigma(z_{i,c})$. Nhãn $y_{i,c} \in \{0, 1\}$. BCE trung bình trên N_a ô:

$$\mathcal{L}_{\text{cls}} = -\frac{1}{N_a} \sum_{i=1}^{N_a} \sum_{c=1}^{N_c} \left[y_{i,c} \log \hat{p}_{i,c} + (1 - y_{i,c}) \log(1 - \hat{p}_{i,c}) \right].$$

- Hộp** (ví dụ CloU-style): trên mỗi cặp góc $m_{ik} = 1$, $\ell_{\text{box}}(b_i, b_k^{\text{gt}}) = 1 - \text{CIoU}(\cdot)$; $\mathcal{L}_{\text{box}}$ là trung bình các ℓ_{box} trên các mẫu dương.
- DFL**: mỗi cạnh có softmax $\hat{p}_j$ ($j=0, \dots, L-1$, $L=\text{reg_max}$). Offset liên tục y (suy ra từ GT) không rơi trọn vào một bin: đặt $y^- = \lfloor y \rfloor$, $y^+ = \lceil y \rceil$ — nhãn *mềm* đặt khối lượng trên hai bin *cận dưới* / *cận trên*:

$$w^- = y^+ - y, \quad w^+ = y - y^- \quad (y^- < y^+).$$

Loss một cạnh (cross-entropy với nhãn mềm đó):

$$\mathcal{L}_{\text{dfl}}^{(\text{cnh})} = -\left(w^- \log \hat{p}_{y^-} + w^+ \log \hat{p}_{y^+} \right).$$

Khi y nguyên ($y^- = y^+ = y$): $\mathcal{L}_{\text{dfl}}^{(\text{cnh})} = -\log \hat{p}_y$. Cộng l, t, r, b trên các ô có góc dương.

Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

31 Loss

YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

DFL — minh họa nhãn mềm (hai bin cận)

Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

32 Loss

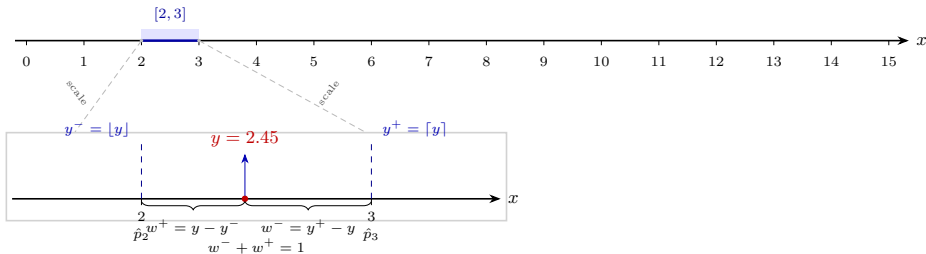
YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết



$$\mathcal{L}_{\text{df}} = -(w^- \log \hat{p}_{y^-} + w^+ \log \hat{p}_{y^+}) = -(w^- \log \hat{p}_2 + w^+ \log \hat{p}_3).$$

Đọc hình (một cạnh / một chiều offset)

Trên: trực tổng quan các bin $0, \dots, L-1$ (chỉ ghi chỉ số); đoạn $[2, 3]$ được tô để nhận diện.
Dưới: hai đường dóng (nét đứt) nối mép đoạn đó xuống khung *phóng đại* — khoảng $[1.5, 3.5]$ được kéo giãn ngang để vẽ y^-, y^+, y và nẹp w^-, w^+ không chồng lấp. Offset liên tục y (ví dụ 2,45) đặt nhãn mềm: $w^- = y^+ - y$, $w^+ = y - y^-$, $w^- + w^+ = 1$; loss một cạnh là CE có trọng số như công thức ngay trong hình.

Từ kiến trúc chung đến YAML của Ultralytics

- Các slide trước đã mô tả kiến trúc chung của họ YOLO một giai đoạn:

Image $\rightarrow$ Backbone $\rightarrow$ Neck $\rightarrow$ Head $\rightarrow$ NMS

- Trong cấu hình YAML của Ultralytics:
 - phần backbone: mô tả các tầng trích xuất đặc trưng chính;
 - phần head: thường bao gồm cả **Neck** FPN/PAN và lớp Detect cuối cùng.

- Vì vậy, khi đọc YAML, cần tách về mặt tư duy:

head: in YAML $\approx$ Neck + Detection Head

- YOLOv8 và YOLO11 giống nhau ở triết lý detection head: anchor-free, DFL cho box regression, sigmoid/BCE cho classification. Khác biệt chính nằm ở các khối xử lý đặc trưng trong backbone và neck.

Cách đọc đúng

Không nên chỉ đọc YAML theo tên backbone/head; nên ánh xạ lại thành ba module học thuật: **Backbone – Neck – Head**.

Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

33 YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

YOLOv8 vs YOLO11 — nhìn theo ba module



Module	YOLOv8	YOLO11
Backbone	Conv + C2f + SPPF	Conv + C3k2 + SPPF + C2PSA
Neck	FPN/PAN với C2f làm khối tinh chỉnh	FPN/PAN với C3k2 làm khối tinh chỉnh
Head	Decoupled Detect: box branch + class branch	Về nguyên lý giống YOLOv8: box branch + class branch
Box regression	DFL: $4 \times N_r$ kênh	DFL: $4 \times N_r$ kênh
Classification	N_c logits, sigmoid + BCE	N_c logits, sigmoid + BCE

Kết luận nhanh

YOLO11 không thay đổi triết lý head của YOLOv8; nó chủ yếu thay khối trích xuất/tinh chỉnh đặc trưng để đạt hiệu quả tốt hơn.

Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

34 YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

YOLOv8 — Backbone

- Backbone của YOLOv8 dùng các khối chính:

Conv, C2f, SPPF

- Conv: giảm kích thước không gian và tăng số kênh đặc trưng.
- C2f: khối CSP-style giúp tăng khả năng biểu diễn nhưng vẫn giữ chi phí tính toán hợp lý.
- SPPF: tổng hợp ngữ cảnh ở tầng sâu bằng pooling nhanh.

Các mức đặc trưng backbone

P_3 : stride 8, P_4 : stride 16, P_5 : stride 32

Ý nghĩa

YOLOv8 backbone tạo ra ba mức đặc trưng thô P_3, P_4, P_5 để đưa vào neck.



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

35 YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

YOLOv8 — Neck

- Neck của YOLOv8 dùng topology kiểu **FPN + PAN**.
- FPN truyền ngữ nghĩa từ tầng sâu xuống tầng nông:

$$P_5 \rightarrow P_4 \rightarrow P_3$$

- PAN truyền thông tin vị trí/chi tiết từ tầng nông ngược lên tầng sâu:

$$P_3 \rightarrow P_4 \rightarrow P_5$$

- Các khối tinh chỉnh trong neck chủ yếu là C2f.

Ký hiệu trong bài giảng

$$P_3, P_4, P_5 \implies P_3^*, P_4^*, P_5^*$$

Ý nghĩa

P_3^*, P_4^*, P_5^* là đặc trưng sau neck: cân bằng hơn giữa chi tiết cục bộ và ngữ cảnh toàn cục.



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

36 YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

YOLOv8 — Detection Head



- YOLOv8 dùng **decoupled head**: tách riêng hai nhánh dự đoán:
 - box branch**: dự đoán vị trí bounding box;
 - class branch**: dự đoán logit phân loại.
- Mỗi nhánh thường gồm các khối **Conv-BN-SiLU** trước khi đi tới lớp Conv2d cuối.
- Box branch:
 - xuất ra $4 \times N_r$ kênh;
 - dùng **DFL** để dự đoán phân phối khoảng cách cho l, t, r, b .
- Class branch:
 - xuất ra N_c logits;
 - dùng **sigmoid + BCE** cho huấn luyện phân loại.

$$\text{YOLOv8 Head: Feature} \rightarrow \begin{cases} \text{Conv blocks} \rightarrow 4N_r \text{ box logits} \\ \text{Conv blocks} \rightarrow N_c \text{ class logits} \end{cases}$$

Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

37 YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

YOLO11 — Backbone



- YOLO11 giữ triết lý backbone nhiều mức như YOLOv8, nhưng thay đổi khối xử lý chính.

- Các khối chính:

Conv, C3k2, SPPF, C2PSA

- C3k2: thay vai trò của C2f, hướng tới biểu diễn hiệu quả hơn.
- SPPF: vẫn dùng để tổng hợp ngữ cảnh ở tầng sâu.
- C2PSA: bổ sung cơ chế chú ý để tăng khả năng chọn lọc đặc trưng.

Điểm khác YOLOv8

YOLOv8: C2f $\Rightarrow$ YOLO11: C3k2 + C2PSA

Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

38 YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

YOLO11 — Neck

- YOLO11 vẫn dùng topology neck kiểu **FPN + PAN**.
- Luồng xử lý vẫn là:

top-down fusion + bottom-up aggregation

- Khác biệt chính: các khối tinh chỉnh trong neck chuyển từ C2f sang C3k2.

Module	YOLOv8	YOLO11
FPN/PAN topology	Giữ nguyên ý tưởng	Giữ nguyên ý tưởng
Feature refiner	C2f	C3k2
Output to head	P_3^*, P_4^*, P_5^*	P_3^*, P_4^*, P_5^*

Ý nghĩa

YOLO11 không đổi vai trò của neck; nó thay khối xử lý để neck tinh luyện đặc trưng hiệu quả hơn.



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

39 YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

YOLO11 — Detection Head

- YOLO11 vẫn giữ triết lý head tách nhánh:
 - box branch
 - class branch
- Nhưng implementation được tối ưu bằng **Depthwise + Pointwise Conv (DW/PW)**.
- DWConv: mỗi channel tự học spatial filter riêng.
- PWConv (1x1): trộn thông tin giữa các channel.
- Lợi ích:
 - ít tham số hơn Conv chuẩn
 - FLOPs thấp hơn
 - nhanh hơn trên edge GPU/mobile

Head block: Conv $\rightarrow$ DWConv + PWConv



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

40 YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Khác biệt cốt lõi giữa YOLOv8 và YOLO11

- Không khác ở triết lý detection:
 - đều là one-stage detector;
 - đều dùng $P_3/P_4/P_5$;
 - đều dùng FPN/PAN neck;
 - đều dùng DFL cho box regression;
 - đều dùng sigmoid/BCE cho classification.
- Khác chủ yếu ở khối biểu diễn đặc trưng:
 - YOLOv8 dùng C2f;
 - YOLO11 dùng C3k2;
 - YOLO11 thêm C2PSA sau SPPF ở backbone.

Thông điệp

YOLOv8 và YOLO11 có cùng “xương sống tư duy” về detection head; YOLO11 cải tiến cách trích xuất và tinh luyện feature trước khi đưa vào head.

Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

41 YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Tóm tắt: đọc YOLOv8/YOLO11 theo module



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Dữ liệu và nhãn

Yolo: Ý tưởng

Kích thước tensor

Chi tiết đầu phát hiện

TaskAlignedAssigner

Loss

42 YOLOv8 vs YOLO11

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Câu hỏi	YOLOv8	YOLO11
Backbone dùng khối gì?	C2f + SPPF	C3k2 + SPPF + C2PSA
Neck làm gì?	FPN/PAN, tinh chỉnh bằng C2f	FPN/PAN, tinh chỉnh bằng C3k2
Head xuất gì?	$4N_r + N_c$ mỗi điểm	$4N_r + N_c$ mỗi điểm
Box regression?	DFL	DFL
Classification?	sigmoid + BCE	sigmoid + BCE

Thông điệp

Khi dạy YOLO, nên tách rõ: kiến trúc học thuật là Backbone–Neck–Head; còn YAML của Ultralytics là cách cài đặt cụ thể.

Bước suy diễn (Inference)

Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

43 Bước suy diễn

Quy trình suy diễn

NMS

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Quy trình suy diễn (tổng quát)



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

44 Quy trình suy diễn

NMS

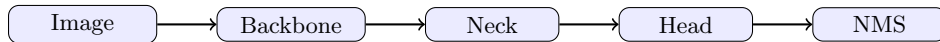
Đánh giá kết quả

Mô hình nâng cao

Tổng kết

- **Tiền xử lý:** ảnh $\rightarrow$ resize/letterbox về imgsz và chuẩn hóa giống pipeline huấn luyện Ultralytics.
- **Forward:** Backbone–Neck–Head $\Rightarrow$ dự đoán trên N_a vị trí (kim tự tháp P_3, P_4, P_5): logits cls và loc như các slide trước.
- **Giải mã:** đưa loc (DFL) về hộp trong tọa độ ảnh đầu vào; lấy conf và lớp (argmax trên cls sau sigmoid, theo cấu hình mô hình).
- **Lọc:** bỏ các dự đoán có conf dưới ngưỡng; giới hạn số ứng viên tối đa (max det) nếu cấu hình.
- **Hậu xử lý:** NMS theo lớp (slide sau); ánh xạ tọa độ ngược về ảnh gốc khi đã letterbox.

Chuỗi xử lý (minh họa)



Pipeline suy diễn điển hình: ảnh $\rightarrow$ Backbone–Neck–Head $\rightarrow$ lọc conf và **NMS**.

Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

45 Quy trình suy diễn

NMS

Đánh giá kết quả

Mô hình nâng cao

Tổng kết

Minh họa tính toán NMS



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

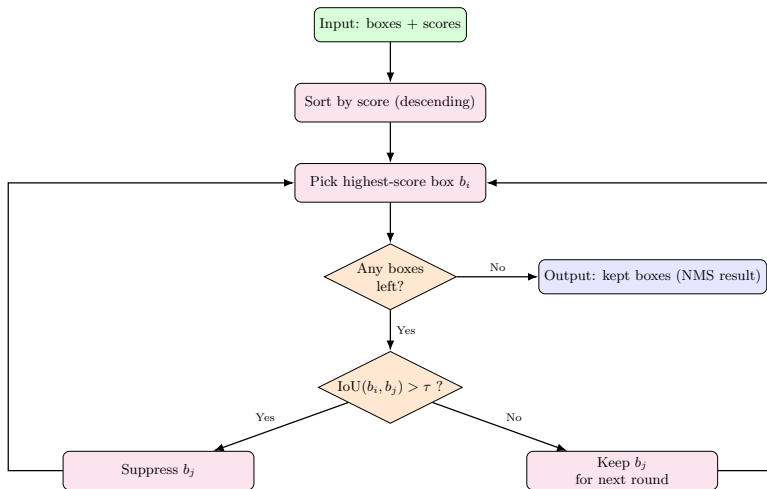
Quy trình suy diễn

46 NMS

Đánh giá kết quả

Mô hình nâng cao

Tổng kết



NMS từng bước (1/5)

Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

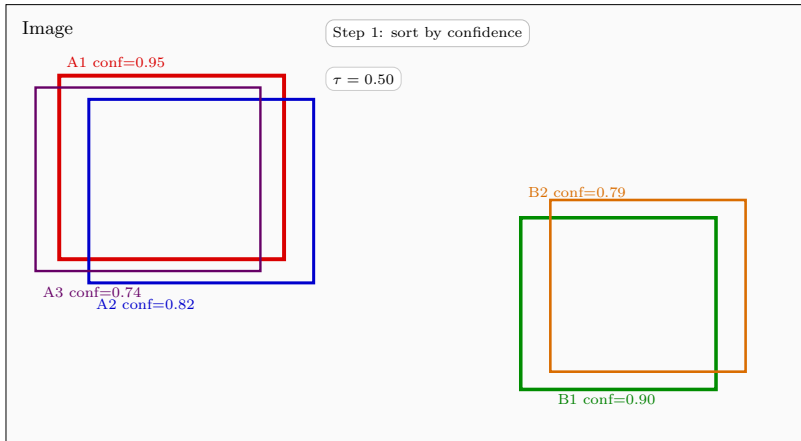
Quy trình suy diễn

47 NMS

Đánh giá kết quả

Mô hình nâng cao

Tổng kết



NMS từng bước (2/5)

Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

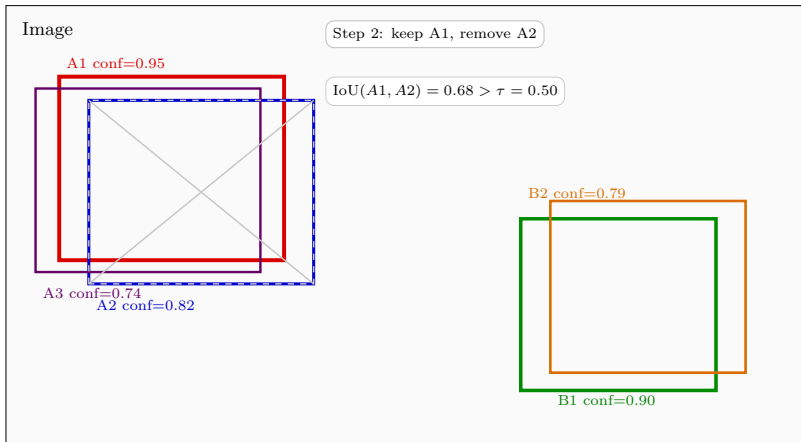
Quy trình suy diễn

48 NMS

Đánh giá kết quả

Mô hình nâng cao

Tổng kết



NMS từng bước (3/5)

Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Quy trình suy diễn

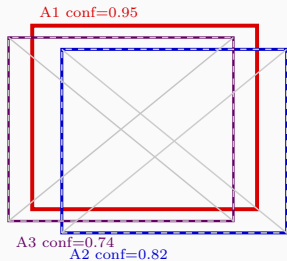
49 NMS

Đánh giá kết quả

Mô hình nâng cao

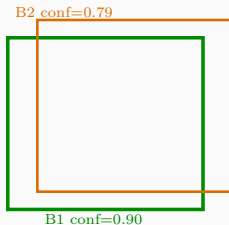
Tổng kết

Image



Step 3: keep A1, remove A3

$\text{IoU}(A1, A3) = 0.73 > \tau = 0.50$



NMS từng bước (4/5)

Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Quy trình suy diễn

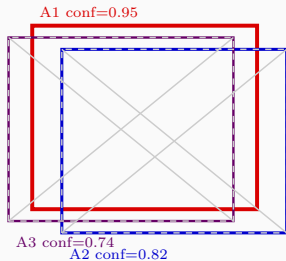
50 NMS

Đánh giá kết quả

Mô hình nâng cao

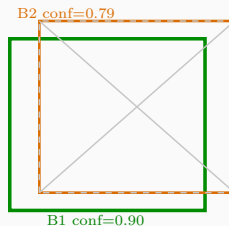
Tổng kết

Image



Step 4: keep B1, remove B2

$\text{IoU}(B1, B2) = 0.64 > \tau = 0.50$



NMS từng bước (5/5)

Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

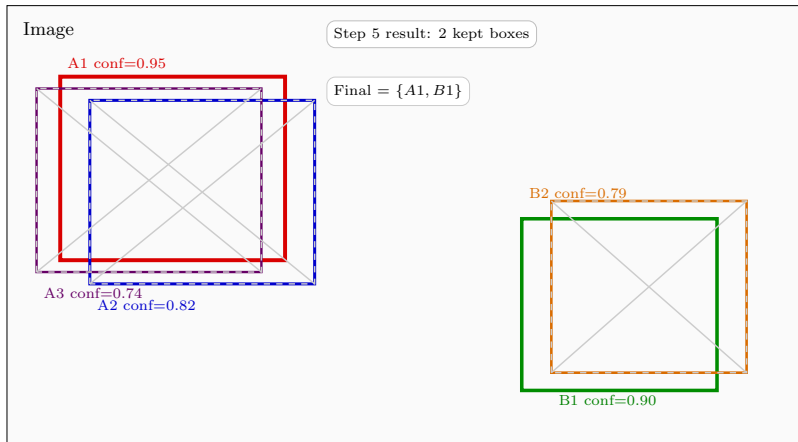
Quy trình suy diễn

51 NMS

Đánh giá kết quả

Mô hình nâng cao

Tổng kết



Đánh giá kết quả

Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

52 Đánh giá kết
quả

Khớp và đếm

Precision Recall

AP mAP

Lỗi thường gặp

Dữ liệu

Pipeline

Mô hình nâng cao

Tổng kết

Đánh giá kết quả: lộ trình

- 1 So khớp dự đoán với ground-truth bằng IoU.
- 2 Đếm TP / FP / FN.
- 3 Tính Precision / Recall.
- 4 Vẽ PR Curve.
- 5 Tính AP và mAP.
- 6 Kiểm tra lỗi dữ liệu và pipeline đánh giá.

Prediction Quality + Localization Quality = Detection Performance



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

53 Đánh giá kết
quả

Khớp và đếm

Precision Recall

AP mAP

Lỗi thường gặp

Dữ liệu

Pipeline

Mô hình nâng cao

Tổng kết

IoU: Intersection over Union

$$\text{IoU} = \frac{|B_{\text{pred}} \cap B_{\text{gt}}|}{|B_{\text{pred}} \cup B_{\text{gt}}|}$$

- Đo mức chồng lấn giữa box dự đoán và box thật.
- IoU càng lớn $\rightarrow$ định vị càng chính xác.
- Nếu cùng lớp và $\text{IoU} \geq \tau$ thì xem là khớp.

Ví dụ:

$$\tau = 0.5$$



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

54 Khớp và đếm

Precision Recall

AP mAP

Lỗi thường gặp

Dữ liệu

Pipeline

Mô hình nâng cao

Tổng kết

TP, FP, FN

- **TP (True Positive):** đúng lớp + IoU đạt ngưỡng.
- **FP (False Positive):** sai lớp, box sai, hoặc box trùng lặp.
- **FN (False Negative):** vật thật nhưng model bỏ sót.

Good detector = many TP + few FP + few FN



Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

55 Khớp và đếm

Precision Recall

AP mAP

Lỗi thường gặp

Dữ liệu

Pipeline

Mô hình nâng cao

Tổng kết

Precision và Recall

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

- Precision cao: ít báo nhầm.
- Recall cao: ít bỏ sót.
- Thường phải đánh đổi giữa hai đại lượng.



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Khớp và đếm

56 Precision Recall

AP mAP

Lỗi thường gặp

Dữ liệu

Pipeline

Mô hình nâng cao

Tổng kết

PR Curve

- Quét nhiều ngưỡng confidence.
- Mỗi ngưỡng tạo ra một cặp:

(Recall, Precision)

- Nối các điểm lại thành PR Curve.

Curve càng cao càng tốt



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Khớp và đếm

57 Precision Recall

AP mAP

Lỗi thường gặp

Dữ liệu

Pipeline

Mô hình nâng cao

Tổng kết

Average Precision (AP)

- AP = diện tích dưới đường cong PR.
- Tính cho một lớp.

$$AP = \int_0^1 P(R) dR$$

Ví dụ:

- $AP_{50} : IoU = 0.50$
- $AP_{75} : IoU = 0.75$



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Khớp và đếm

Precision Recall

58 AP mAP

Lỗi thường gặp

Dữ liệu

Pipeline

Mô hình nâng cao

Tổng kết

Mean Average Precision (mAP)

$$mAP = \frac{1}{N_c} \sum_{c=1}^{N_c} AP_c$$

- Trung bình AP trên tất cả lớp.
- Metric chuẩn cho object detection.

COCO thường báo:

- mAP@50
- mAP@50:95



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Khớp và đếm

Precision Recall

59 AP mAP

Lỗi thường gặp

Dữ liệu

Pipeline

Mô hình nâng cao

Tổng kết

Sai sót khi đánh giá

- Train và validation dùng resize khác nhau.
- Sai mapping class-id.
- Có rò rỉ dữ liệu giữa train / val.
- NMS config khác baseline.
- Label sai hoặc box quá lỏng.

Bad Evaluation $\Rightarrow$ Bad Conclusion



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Khớp và đếm

Precision Recall

AP mAP

60 Lỗi thường gặp

Dữ liệu

Pipeline

Mô hình nâng cao

Tổng kết

Định dạng dữ liệu phổ biến

- **YOLO:**
class cx cy w h
- **COCO:**
JSON annotations
- **VOC:**
XML per image
- Cần chuyển đổi đúng format trước khi train/eval.



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Khớp và đếm

Precision Recall

AP mAP

Lỗi thường gặp

61 Dữ liệu

Pipeline

Mô hình nâng cao

Tổng kết

Đối tượng nhỏ và mất cân bằng lớp

- Small objects: ít pixel, khó detect.
- Rare classes: ít dữ liệu huấn luyện.
- Giải pháp:
 - tăng resolution
 - augment hợp lý
 - re-sampling
 - thêm dữ liệu



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Khớp và đếm

Precision Recall

AP mAP

Lỗi thường gặp

62 Dữ liệu

Pipeline

Mô hình nâng cao

Tổng kết

Augmentation và Throughput

Augmentation

- Flip
- HSV
- Mosaic
- MixUp

Throughput

- Loader chậm làm GPU chờ dữ liệu.
- Tối ưu:
 - workers
 - cache
 - fast storage



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Khớp và đếm

Precision Recall

AP mAP

Lỗi thường gặp

Dữ liệu

63 Pipeline

Mô hình nâng cao

Tổng kết



Mô hình nâng cao

Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

64 Mô hình nâng
cao

YOLOv26

DETR

Hungarian Matching

Tổng kết

YOLOv26: Tổng quan kiến trúc



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

65 YOLOv26

DETR

Hungarian Matching

Tổng kết

Input $\rightarrow$ Backbone $\rightarrow$ Neck $\rightarrow$ Dual Detect Heads

- YOLOv26 vẫn giữ khung chuẩn của họ YOLO:
 - Backbone**: trích đặc trưng nhiều mức P3, P4, P5
 - Neck**: hợp nhất đa tỷ lệ (FPN/PAN)
 - Head**: dự báo box + class
- Điểm đổi mới nằm ở **Detection Head**.
- Thay vì một head duy nhất, YOLOv26 dùng:

One-to-Many Head + One-to-One Head

- Hai head cùng dùng feature từ Neck.

YOLOv26: Hai Head gần như clone cùng kiến trúc



- Hai nhánh detection head có cấu trúc gần như giống nhau:
 - cùng input feature maps P3,P4,P5
 - cùng feature refiner conv blocks
 - cùng box branch
 - cùng class branch
 - cùng decode box theo kiểu YOLO hiện đại
- Khác biệt chính không nằm ở kiến trúc mạng, mà ở:
 - cách gán nhãn (assignment)
 - cách chọn positive samples
 - loss supervision
 - cách dùng khi inference

Same Network Structure $\neq$ Same Training Target

Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

66 YOLOv26

DETR

Hungarian Matching

Tổng kết

YOLOv26: Vai trò từng nhánh

One-to-Many Head

- Một GT object gán cho nhiều candidates.
- Dense supervision kiểu YOLO truyền thống.
- Học nhanh, recall tốt.

One-to-One Head

- Một GT object ghép với một prediction tốt nhất.
- Output sạch hơn, ít duplicate.
- Hướng tới giảm phụ thuộc NMS.

Thực tế sử dụng

- Train: dùng cả hai head.
- Inference: thường ưu tiên head one-to-one.

Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

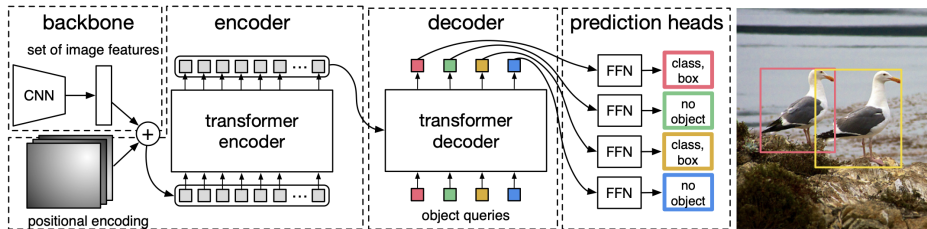
67 YOLOv26

DETR

Hungarian Matching

Tổng kết

DETR: Ý tưởng tổng quan (1/2)



Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

YOLOv26

68 DETR

Hungarian Matching

Tổng kết

Detr - ý chính

- DETR dùng encoder-decoder Transformer để dự đoán trực tiếp tập các đối tượng.
- Mỗi object query tạo ra một dự đoán class + box theo cơ chế one-to-one matching.
- Cách tiếp cận này giảm phụ thuộc vào anchor và heuristic hậu xử lý.

DETR: Ý tưởng tổng quan (2/2)

- Không quét hàng nghìn anchor/grid points như YOLO.
- Dùng số lượng nhỏ **object queries** học được, ví dụ:

$$N_q = 100$$

- Mỗi query dự đoán:
 - một class
 - một bounding box
 - hoặc no-object
- Mục tiêu:
 - Giảm duplicate predictions
 - Giảm phụ thuộc NMS
 - **Huấn luyện end-to-end**



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

YOLOv26

69 DETR

Hungarian Matching

Tổng kết

DETR: Tensor view (1/3)



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

YOLOv26

70 DETR

Hungarian Matching

Tổng kết

(1) Rút trích đặc trưng ảnh

- Biểu thức: $I = \text{CNN}(\text{Images})$.
- + **CNN**: backbone CNN-based (ResNet, ConvNeXt, ...).
- + Images : $(B, 3, H_i, W_i)$.
- + $I : (B, C, H, W)$.

(2) Phẳng hóa và cộng vị trí

- Biểu thức: $X = \text{Flatten}(I) + PE$.
- + $\text{Flatten}(I) : (B, HW, C)$.
- + $PE : (1, HW, C)$ (hoặc broadcast theo batch).
- + $X : (B, HW, C)$.

DETR: Tensor view (2/3)



(3) Encoder

- Biểu thức: $F = \text{Encoder}(X, X, X)$.
- + $X : (B, HW, C) \rightarrow F : (B, HW, C)$.
- + Ý nghĩa: gom ngữ cảnh toàn cục bằng self-attention.

(4) Decoder với object queries

- Biểu thức: $Q_f = \text{Decoder}(Q_0, F, F)$.
- + Q_0 gốc là tham số học được qua nn.Embedding: (N_q, C) ; sau broadcast/expand theo batch mới thành (B, N_q, C) .
- + $Q_f : (B, N_q, C)$ là embedding theo từng object query.

(5) Head dự đoán class + box

- Biểu thức: $(\hat{p}, \hat{b}) = \text{Heads}(Q_f)$.
- + $\hat{p} : (B, N_q, N_c + 1)$ (có lớp no-object).
- + $\hat{b} : (B, N_q, 4)$ (box chuẩn hóa).

Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

YOLOv26

71 DETR

Hungarian Matching

Tổng kết

DETR: Tensor view (3/3)



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

YOLOv26

72 DETR

Hungarian Matching

Tổng kết

(6) Hungarian matching

- Biểu thức: $\sigma^* = \text{Hungarian}(\mathcal{Y}, \hat{\mathcal{Y}})$.
- + Gán 1-1 giữa GT và predicted queries.
- + Query dư được gán no-object.

(7) Loss huấn luyện

- Biểu thức:

$$\mathcal{L} = \mathcal{L}_{cls} + \lambda_1 \|b - \hat{b}_{\sigma^*}\|_1 + \lambda_g \mathcal{L}_{giou}.$$

- + Tổng hợp classification + box regression + GloU.
- + Huấn luyện end-to-end, không cần gán anchor heuristic.

DETR: Inference gồm các bước nào?

- (1) **Tiền xử lý ảnh**: resize/normalize theo cấu hình model.
- (2) **Backbone + Encoder**: lấy đặc trưng ảnh và mã hóa ngữ cảnh toàn cục.
- (3) **Decoder với queries**: cho N_q object queries truy vấn bộ nhớ ảnh để tạo embedding từng object.
- (4) **Head dự đoán**: mỗi query sinh class logits (có no-object) và box chuẩn hóa.
- (5) **Lọc dự đoán**: bỏ các query có điểm thấp/no-object, giữ top-k theo score.
- (6) **Hậu xử lý tọa độ**: đổi box từ hệ chuẩn hóa về kích thước ảnh gốc (undo resize/padding nếu có).

Lưu ý

Inference của DETR thường không cần NMS kiểu YOLO do cơ chế one-to-one được học trong huấn luyện.

Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

YOLOv26

73 DETR

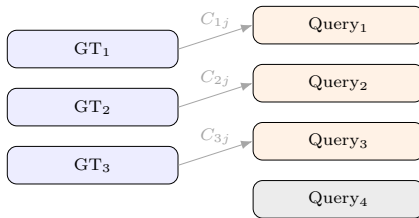
Hungarian Matching

Tổng kết

Hungarian Matching (1/5): Input và ký hiệu

- **Ground-truth:** $\mathcal{Y} = \{y_i\}_{i=1}^{N_{gt}}$, mỗi $y_i = (c_i, b_i)$.
- **Predictions:** $\hat{\mathcal{Y}} = \{\hat{y}_j\}_{j=1}^{N_q}$, mỗi $\hat{y}_j = (\hat{p}_j, \hat{b}_j)$.
- Thường $N_q > N_{gt}$, query dư sẽ về no-object.
- Ma trận chi phí:

$$C_{ij} = C_{ij}^{cls} + \lambda_1 C_{ij}^{L1} + \lambda_g C_{ij}^{giou}.$$



⇒ tìm phép gán 1-1
chi phí nhỏ nhất

Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

YOLOv26

DETR

74 Hungarian Matching

Tổng kết

Hungarian Matching (2/5): Lập ma trận chi phí

$$C = \begin{bmatrix} 0.32 & 0.81 & 0.44 & 0.96 \\ 0.77 & 0.25 & 0.58 & 0.69 \\ 0.66 & 0.49 & 0.21 & 0.73 \end{bmatrix}$$

GT₁

GT ₁	0.32	0.81	0.44	0.96
GT ₂	0.77	0.25	0.58	0.69
GT ₃	0.66	0.49	0.21	0.73

GT₂

GT₃

Q₁ Q₂ Q₃ Q₄

Các số đỏ: ứng viên tốt của từng hàng

- Hàng = GT, cột = query.
- Mỗi phần tử C_{ij} được tính từ dự đoán của query j so với GT i :

$$C_{ij} = \underbrace{-\log \hat{p}_j(c_i)}_{\text{class cost}} + \lambda_1 \underbrace{\|b_i - \hat{b}_j\|_1}_{\text{L1 box cost}} + \lambda_g \underbrace{\mathcal{L}_{giou}(b_i, \hat{b}_j)}_{\text{GloU cost}}.$$

- Tức là: class càng đúng và box càng gần GT $\Rightarrow C_{ij}$ càng nhỏ.
- Mục tiêu: chọn 1 cột cho mỗi hàng, không trùng cột.

Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

YOLOv26

DETR

75 Hungarian Matching

Tổng kết

Hungarian Matching (3/5): Ràng buộc one-to-one

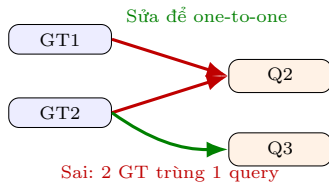


- Nếu chọn từng hàng độc lập, có thể **đụng cột** (nhiều GT cùng chọn một query).

- Hungarian ép ràng buộc:

$$\forall i \neq i', \sigma(i) \neq \sigma(i').$$

- Tức là: mỗi GT ghép đúng một query, mỗi query ghép tối đa một GT.



Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

YOLOv26

DETR

76 Hungarian Matching

Tổng kết

Hungarian Matching (4/5): Nghiệm tối ưu toàn cục



$$\sigma^* = \arg \min_{\sigma \in \mathfrak{S}_{N_q}} \sum_{i=1}^{N_{gt}} C_{i, \sigma(i)}$$

- Tối ưu **tổng chi phí** thay vì tối ưu cục bộ từng hàng.
- Ví dụ chọn: $GT_1 \rightarrow Q_1$, $GT_2 \rightarrow Q_2$, $GT_3 \rightarrow Q_3$.
- Query còn lại (Q_4) để học no-object.

GT ₁	0.32			
GT ₂		0.25		
GT ₃			0.21	
	Q ₁	Q ₂	Q ₃	Q ₄

Ô xanh: matching
được chọn

Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

YOLOv26

DETR

77 Hungarian Matching

Tổng kết

Hungarian Matching (5/5): Kết quả cuối và liên kết loss



- Sau khi có σ^* , DETR chỉ tính loss trên các cặp đã ghép:

$$\mathcal{L}_{match} = \sum_{i=1}^{N_{gt}} \left(\mathcal{L}_{cls}(c_i, \hat{p}_{\sigma^*(i)}) + \lambda_1 \|b_i - \hat{b}_{\sigma^*(i)}\|_1 + \lambda_g \mathcal{L}_{giou}(b_i, \hat{b}_{\sigma^*(i)}) \right).$$

- Các query không ghép GT nhận nhãn no-object trong nhánh classification.
- Hệ quả: mô hình học dự đoán **sạch, one-to-one**, giảm duplicate và thường không cần NMS lúc inference.

Thông điệp

Hungarian matching là cầu nối giữa dự đoán set-based của DETR và hàm loss end-to-end.

Phát hiện đối tượng trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

YOLOv26

DETR

78 Hungarian Matching

Tổng kết

Tổng kết

Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

79 Tổng kết

Tổng kết: Object Detection



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

80 Tổng kết

- Object detection không chỉ trả lời **vật thể là gì**, mà còn phải trả lời **vật thể ở đâu**.
- Output của detector là một tập các dự đoán:

$$\hat{\mathcal{Y}} = \{(\hat{b}_i, \hat{c}_i, \hat{s}_i)\}_{i=1}^N$$

- Mỗi dự đoán gồm:
 - bounding box;
 - class label;
 - confidence score.
- Khó khăn chính: nhiều vật thể, nhiều tỉ lệ, che khuất, nhiễu nền, duplicate boxes.

Tổng kết: tư duy kiến trúc YOLO

- Họ YOLO nên được nhìn theo ba module chính:

Backbone $\rightarrow$ Neck $\rightarrow$ Head

- Backbone:** trích xuất đặc trưng đa mức:

$$P_3, P_4, P_5$$

- Neck:** hợp nhất đặc trưng đa tỷ lệ bằng FPN/PAN:

$$P_3, P_4, P_5 \rightarrow P_3^*, P_4^*, P_5^*$$

- Head:** tại mỗi điểm tham chiếu, dự đoán:

$$\underbrace{4N_r}_{\text{box / DFL}} + \underbrace{N_c}_{\text{class logits}}$$

Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

81 Tổng kết

Tổng kết: YOLOv8, YOLO11, YOLOv26



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

82 Tổng kết

Model	Ý chính
YOLOv8	Backbone/Neck dùng C2f; head tách box branch và class branch; box regression dùng DFL.
YOLO11	Giữ cùng triết lý detection nhưng cải tiến khối biểu diễn: C3k2, C2PSA, và head tối ưu hơn bằng DW/PW conv.
YOLOv26	Mở rộng theo hướng dual heads: one-to-many để học tốt và one-to-one để suy diễn sạch hơn, giảm phụ thuộc NMS.

Thông điệp

Các phiên bản YOLO khác nhau ở thiết kế block và chiến lược huấn luyện, nhưng vẫn nên được đọc theo trục Backbone–Neck–Head.

Tổng kết: suy diễn và đánh giá



Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

83 Tổng kết

- Quy trình suy diễn:

Image $\rightarrow$ Forward $\rightarrow$ Decode boxes $\rightarrow$ Confidence filter $\rightarrow$ NMS $\rightarrow$ Final detections

- NMS dùng để loại bỏ các box trùng lặp mạnh.
- Đánh giá detection dựa trên:
 - IoU;
 - TP / FP / FN;
 - Precision / Recall;
 - AP / mAP.
- mAP chỉ có ý nghĩa khi dữ liệu, nhãn, split và pipeline đánh giá được kiểm soát đúng.

Điều cần nhớ sau bài học



- Đừng học YOLO chỉ bằng tên phiên bản; hãy học bằng cấu trúc:

data → feature pyramid → head → decode → evaluation

- Khi đọc source code YOLO:
 - đọc file YAML để hiểu graph mô hình;
 - đọc block.py để hiểu backbone/neck blocks;
 - đọc head.py để hiểu Detect head;
 - đọc loss/eval để hiểu cách huấn luyện và đo mAP.
- Mục tiêu cuối cùng: không chỉ biết dùng YOLO, mà biết sửa YOLO, phân tích YOLO, và thiết kế detector mới.

Phát hiện đối tượng
trong ảnh

Mục lục

Giới thiệu

Họ Yolo-Detector

Bước suy diễn

Đánh giá kết quả

Mô hình nâng cao

84 Tổng kết